
Networking and Protocols I

Lecture 2 — TCP, TLS 1.3, HTTP/1.1, HTTP/2, HTTP/3 and QUIC, WebSocket, SSE

RECALL FROM WEEK 1

Four Things You Should Already Own

Latency $\neq$ throughput

Throughput can rise while latency has already broken the budget.

The tail is the product

p95 and p99, never the mean. Fan-out turns a rare slow call into a common one.

$L = \lambda \times W$

Concurrency is arrival rate times time in system. Longer calls cost you sockets and threads.

A design needs a number

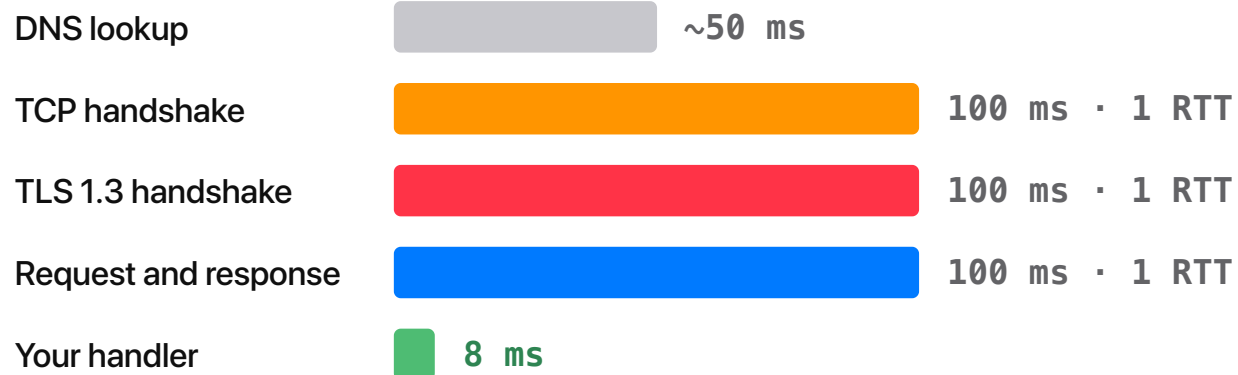
Target RPS, target p99, data volume, cost ceiling. Otherwise there is nothing to defend.

Today we find out who spends that budget. Usually it is not your code.

WHY THIS LECTURE EXISTS

Where the Milliseconds Go

One HTTPS request to a service across a 100 ms link, on a cold connection. The server-side handler is the smallest item on the bill.



Three round trips before the first byte of your JSON. Reuse that connection and the next request costs 108 ms instead of 358 ms — a 3× win with no code change.

TWO HOURS

Today's Route

01

The transport beneath your API

TCP handshake, retransmission, flow and congestion control, why a cold connection is slow.

02

Encryption is not free

TLS 1.2 versus 1.3, resumption and 0-RTT, what it costs in RTTs and in CPU.

03

HTTP, three generations

Keep-alive, multiplexing, head-of-line blocking, QUIC — and when each one is the right answer.

04

Keeping the connection open

Polling, SSE, WebSocket, and what a million open sockets do to a server.

01

The Transport Beneath Your API

Handshake

Retransmission

Flow control

Congestion control

Head-of-line blocking

ORIENTATION

The Stack You Actually Pay For

Application	HTTP/1.1 · HTTP/2 · HTTP/3 · gRPC · WebSocket · SSE	your contract
Security	TLS 1.2 · TLS 1.3 — handshake, certificates, ALPN	1–2 RTT + CPU
Transport	TCP — reliable ordered byte stream · UDP — datagrams, and QUIC on top	1 RTT + windows
Network	IP — routing, fragmentation, MTU (1500 bytes on Ethernet)	propagation delay
Link	Ethernet, Wi-Fi, mobile radio — where loss and jitter are born	loss and jitter

A protocol decision is a decision about how many of these layers you have to cross, and how often.

TRANSPORT

What TCP Promises, and What It Charges

THE PROMISE

Reliable. Lost segments are retransmitted until acknowledged.

Ordered. Bytes are delivered to the application in the order they were sent.

A stream. No message boundaries — your framing is your own problem.

Controlled. It will not overwhelm the receiver or the network.

THE CHARGE

A handshake before the first byte of payload.

Head-of-line blocking: one lost segment stalls everything behind it.

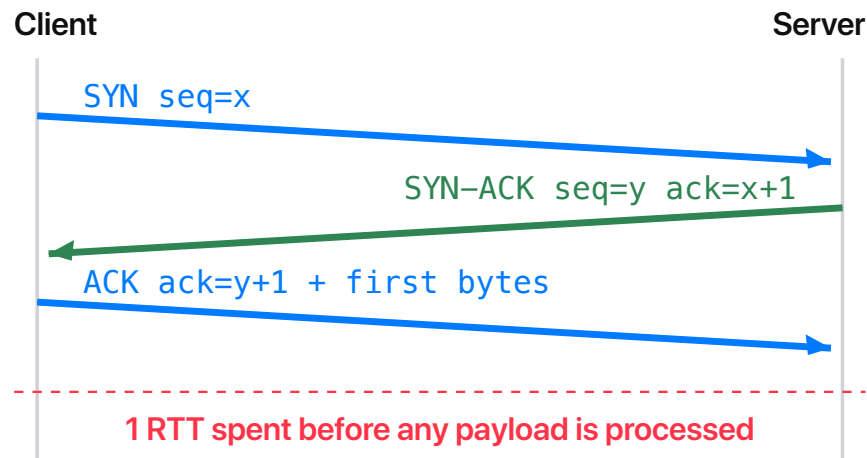
A slow start: the connection does not know the path capacity yet.

Kernel state per connection — buffers, timers, a file descriptor.

Reliability is not a property of the network. It is a loop of timers and acknowledgements, and loops cost time.

CONNECTION SETUP

The Three-Way Handshake



Why three messages

Both sides must agree on a starting sequence number and prove the address is live. Two messages cannot do that.

The backlog queue

Half-open connections sit in a kernel queue. Fill it and new clients are refused — the mechanism behind a SYN flood.

TCP Fast Open

Sends data in the SYN using a cached cookie. Real, standardised, and blocked by enough middleboxes that you cannot rely on it.

ARITHMETIC

What a Cold Connection Costs

PATH	RTT	TCP + TLS 1.3	FIRST RESPONSE
Same rack	0.2 ms	0.4 ms	0.6 ms
Same datacentre	0.5 ms	1 ms	1.5 ms
Almaty ↔ Astana	≈ 20 ms	40 ms	60 ms
Almaty ↔ Frankfurt	≈ 100 ms	200 ms	300 ms
Mobile network, poor signal	100–400 ms	up to 800 ms	> 1 s

Order-of-magnitude figures for a typical fibre path. The pattern is what matters: setup dominates as soon as the distance grows, so a connection you keep is worth more than any micro-optimisation in the handler.

WHERE THE TAIL IS BORN

Loss, Acknowledgement, Retransmission

FAST PATH

Fast retransmit

Three duplicate ACKs for the same byte mean the next segment is missing. The sender resends immediately, without waiting for a timer.

cost ≈ 1 RTT

SLOW PATH

Retransmission timeout

Nothing comes back at all — for example the last segment of a response is lost. The sender waits out the RTO, which starts at 1 s on many stacks and doubles on every further loss.

cost 200 ms – 1 s +

CONSEQUENCE

Your p99 is someone's lost packet

A 1 % loss rate on a path with many round trips per request produces a visible tail. The server-side histogram shows nothing, because the server did its work on time.

Worth knowing by name

SACK — acknowledge the ranges that did arrive, so only the hole is resent. **Tail Loss Probe** — send a probe rather than wait out the RTO. **Karn's algorithm** — do not sample RTT from a retransmitted segment. All of this is your kernel's job; your job is to know it exists when the tail moves.

TWO CONTROL LOOPS

Flow Control and Congestion Control

FLOW CONTROL

Protects the receiver

The receiver advertises a window: "I have this much buffer left." The sender may not exceed it. If the application stops reading the socket, the window shrinks to zero and the sender stalls.

A slow consumer is therefore a network-level problem, not only an application one. This is backpressure, and it reappears in week 4.

CONGESTION CONTROL

Protects the network

Nobody advertises the capacity of the path, so the sender estimates it: increase the rate until there is evidence of congestion, then back off. Loss, or delay, is that evidence.

Reno and CUBIC treat loss as the signal; BBR estimates bandwidth and round-trip time directly and is now common on large edges.

$\text{in flight} \leq \min(\text{rwnd}, \text{cwnd})$

Whichever window is smaller is the one that limits you. Tuning buffers without knowing which one is binding is guesswork.

CONGESTION WINDOW

Slow Start and Back-Off



Doubling every round trip, then one extra segment per round trip, then a multiplicative cut. Additive increase, multiplicative decrease.

Initial window ≈ 10 segments

About 14 KB may be sent in the first round trip. Anything larger needs a second one.

Growth is per round trip

14 KB, 28 KB, 56 KB... so on a 100 ms path reaching 1 MB in flight takes roughly half a second of nothing but warm-up.

Bandwidth-delay product

$$\text{throughput} \approx \text{window} / \text{RTT}$$

A 64 KB window on a 100 ms path caps you at ~640 KB/s, whatever the link speed.

THE FIRST DESIGN RULE OF THE DAY

Never Open a Connection Per Request

A cold connection pays the handshake, the encryption handshake and the slow start. A warm one pays none of them. This is the cheapest win available in any service that calls another service.

Pool, with limits

Size the pool from Little's Law, not by feel: at 500 RPS and 40 ms per call you need ~20 concurrent connections, not 200.

Keep-alive on both ends

An idle timeout shorter on the server than on the client produces reused-but-dead sockets and sporadic request failures.

Watch the socket states

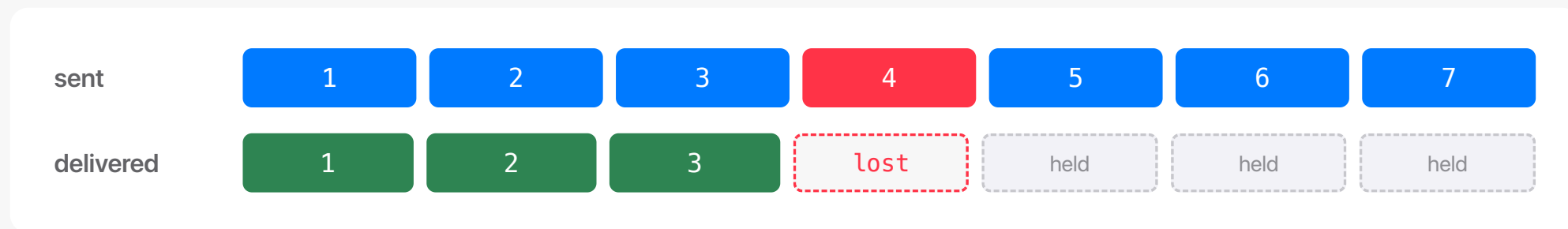
Thousands of sockets in `TIME_WAIT` or a growing accept backlog are load signals your dashboard probably does not show.

In practice today: the same endpoint, hammered with and without connection reuse. Report both p50 and p99 — the gap is the handshake.

REMEMBER THIS TERM

Head-of-Line Blocking

TCP delivers bytes in order. If segment 4 is lost, segments 5, 6 and 7 sit in the kernel buffer — already arrived, already correct — and the application is not allowed to see them.



One lost packet, three delayed responses. Keep this picture: it explains HTTP/1.1, it explains what HTTP/2 could not fix, and it explains why QUIC exists.

SMALL KNOBS, REAL EFFECTS

Four Settings Worth Understanding

TCP_NODELAY

Disables Nagle's algorithm, which batches small writes. Combined with delayed ACK on the other side it can add tens of milliseconds to a small request. Almost every RPC framework sets it.

SO_BACKLOG

How many completed connections may wait for `accept()`. Too small and a traffic spike is refused at the kernel before your code sees it.

SO_KEEPALIVE

Probes an idle connection so a silently dead peer or a NAT that dropped the mapping is detected. Without it a pooled socket can hang until the RTO ladder gives up.

MTU and MSS

1500 bytes on Ethernet, less inside tunnels and overlay networks. A mismatch turns into fragmentation or black-holed packets — a classic Kubernetes networking bug.

The rule: change a kernel setting only after a measurement told you which queue was full. Otherwise you are decorating.

02

Encryption Is Not Free

What TLS guarantees 1.2 versus 1.3 Resumption and 0-RTT The real cost

TLS

Three Guarantees, One Handshake

Confidentiality

Nobody on the path reads the payload.
A symmetric key agreed during the handshake does the actual work.

Integrity

Nobody modifies the payload undetected. Modern suites authenticate and encrypt in one operation (AEAD).

Authentication

You are talking to the host you named.
A certificate chain, validated against a trust store, is what makes this a guarantee.

ONE EXTENSION YOU MUST KNOW BY NAME

ALPN — Application-Layer Protocol Negotiation. The client lists `h2`, `http/1.1` inside the handshake and the server picks one. This is how HTTP/2 is negotiated with no extra round trip, and why HTTP/2 in practice means HTTPS.

HANDSHAKE

TLS 1.2 Needed Two Round Trips. 1.3 Needs One.

TLS 1.2

2 RTT

→ ClientHello

← ServerHello, Certificate,
ServerKeyExchange, Done

→ ClientKeyExchange, ChangeCipherSpec,
Finished

← ChangeCipherSpec, Finished

→ application data

TLS 1.3

1 RTT

→ ClientHello + key_share

← ServerHello + key_share, {Certificate},
{Finished}

→ {Finished}

→ application data

Braces mark encrypted messages: in 1.3 the certificate itself is sent under encryption.

Fewer choices

Legacy ciphers, static RSA key exchange and renegotiation are gone. Forward secrecy is mandatory, not a configuration option.

In numbers, on our 100 ms path

First byte of the response: 400 ms with TLS 1.2, 300 ms with 1.3, 200 ms with resumption, 100 ms with QUIC 0-RTT.

RESUMPTION

Paying the Handshake Once

Session tickets

After a full handshake the server hands the client an encrypted ticket containing the session state. On the next connection the client presents it and the handshake completes in one round trip with no public-key operation.

Behind a load balancer this only works if all nodes share the ticket key. Otherwise every node forces a full handshake and your resumption rate quietly sits at zero.

0-RTT early data

TLS 1.3 lets the client send application data in its very first message, using a key derived from the previous session. The request arrives with the handshake.

Safe: idempotent reads.

Unsafe: anything with a side effect — early data can be captured and replayed, and it has no forward secrecy.

Metric to collect Full handshakes per second versus resumed handshakes per second. If the ratio is not heavily in favour of resumption on a busy edge, you are burning CPU and 100 ms per client for nothing.

TWO SEPARATE BILLS

Latency Cost and CPU Cost

LATENCY — ROUND TRIPS

TCP + TLS 1.2, new session	3 RTT
TCP + TLS 1.3, new session	2 RTT
TCP + TLS 1.3, resumed	2 RTT
QUIC, new session	1 RTT
QUIC, 0-RTT	0 RTT

Counted to the first byte of the request, warm connection excluded.

CPU — WHERE IT ACTUALLY GOES

The handshake is expensive. A signature and a key exchange per full handshake. This is the part that limits connections per second on a TLS terminator.

The bulk stream is cheap. With AES-NI or ChaCha20 on the CPU, encrypting the body costs a few per cent of throughput — not a reason to run plaintext internally.

So the fix is architectural: resume sessions, keep connections alive, and terminate TLS at a layer built for it rather than in every service.

In practice today you will measure this directly: the same load with a fresh handshake per request and with a warm pool. The difference is the slide.

CONFIGURATION THAT MATTERS

Five Things to Set and Watch

01

TLS 1.3 enabled, 1.0 and 1.1 disabled

One round trip saved per connection, and a shorter list of things that can be attacked.

02

Session tickets, shared across nodes

Without a shared key, resumption does not survive the load balancer.

03

ALPN advertising h2

Otherwise your HTTP/2-capable server quietly serves HTTP/1.1 to everyone.

04

A short chain and OCSP stapling

The certificate chain travels in the first round trips; every extra kilobyte competes with the initial congestion window.

05

Certificate expiry alerting

The most embarrassing outage in this course, and still one of the most common in the industry.

LECTURE 2 · PART THREE OF FOUR

03

HTTP, Three Generations

Keep-alive Multiplexing HPACK QUIC When each one wins

GENERATION ONE

HTTP/1.1 — Text, One Request at a Time

1996 · HTTP/1.0

One request, one connection

Connect, ask, answer, close. Every asset on a page paid a full handshake and a fresh slow start.

1997 · KEEP-ALIVE

The connection survives

Persistent connections by default. Requests still strictly one after another, but the handshake and the warm-up are amortised.

PIPELINING

Specified, never usable

Send several requests without waiting — but responses must return in order, so one slow response blocks the queue. Browsers disabled it.

What engineers did instead — and why you still see it

Six connections per origin. Browsers open several sockets to fake concurrency — six times the handshakes, six separate congestion windows.

Domain sharding. Serve assets from several hostnames to lift the limit. More DNS, more handshakes.

Bundling and sprites. Merge many small resources into one big one, because one request is cheaper than fifty.

All three are workarounds for a protocol limitation. HTTP/2 makes all three obsolete — and if you keep them, they make HTTP/2 slower.

THE LIMITATION

One Slow Response Blocks the Rest

HTTP/1.1 · ONE CONNECTION



Request C is trivial and finishes last, because it was behind a slow one. Total time is the sum.

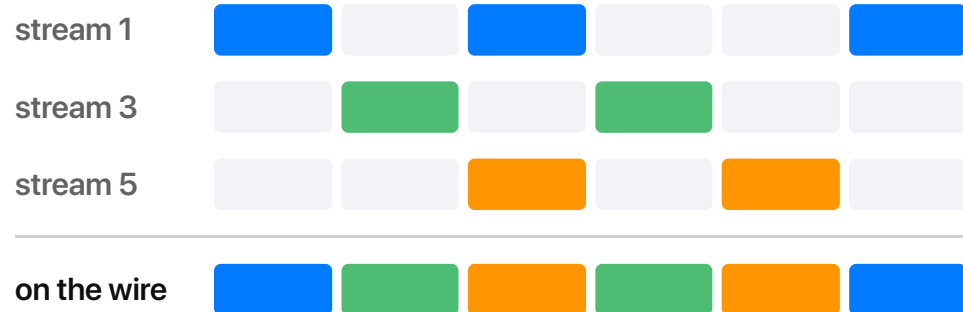
HTTP/2 · ONE CONNECTION, THREE STREAMS



All three start immediately and finish independently. Total time is the maximum, not the sum.

This is application-level head-of-line blocking: caused by the protocol, not by the network. HTTP/2 removes it — but the TCP-level version from part one is still underneath.

HTTP/2 — One Connection, Many Streams



One TCP connection carries interleaved frames belonging to independent streams.

Frame

The unit on the wire. Binary, length-prefixed, typed — HEADERS, DATA, SETTINGS, WINDOW_UPDATE, RST_STREAM.

Stream

One request-response pair, identified by a number. Independent, cancellable, flow-controlled on its own.

Connection

One per origin, carrying hundreds of streams. No sharding, no bundling, no six-socket trick needed.

THE REST OF HTTP/2

Binary Framing, HPACK, and What Died

HPACK header compression

A static table of 61 common header fields, a dynamic table of what this connection has already sent, and Huffman coding for the rest. The second request on a connection can reference a header by index instead of repeating it.

~700 B

typical headers,
HTTP/1.1

tens of B

the same, repeated on a warm h2
connection

The dynamic table is connection state — which is why an HTTP/2 connection is not free to re-establish either.

Kept: stream priorities and flow control

Per-stream windows, and a way to say which stream matters more. Useful, unevenly implemented.

Abandoned: server push

Sending a resource the client did not ask for. It pushed things the client already had cached; browsers removed support.

Why this matters beyond the browser

gRPC is built directly on HTTP/2 streams and framing. Everything here returns in week 3.

THE PROBLEM HTTP/2 COULD NOT SOLVE

Multiplexing on a Single Ordered Stream

All the streams share one TCP connection. TCP delivers bytes in order. So one lost packet stalls every stream in the connection — including the ones whose data has already arrived.

On a clean network

HTTP/2 is strictly better: fewer handshakes, fewer congestion windows, compressed headers, real concurrency. This is the datacentre case, and it is why gRPC chose it.

On a lossy network

One connection means one shared stall. On a mobile link with a few per cent loss, HTTP/2 can measure worse than six independent HTTP/1.1 connections.

The conclusion The fix cannot live in HTTP. It has to live in the transport — which means replacing TCP. That is what QUIC is.

HTTP/3 — HTTP Over QUIC Over UDP

Independent streams

Loss on one stream does not block the others. Ordering is per stream, not per connection — head-of-line blocking finally gone.

Connection migration

A connection ID identifies the session, not the IP and port pair. Switching from Wi-Fi to mobile keeps the connection alive.

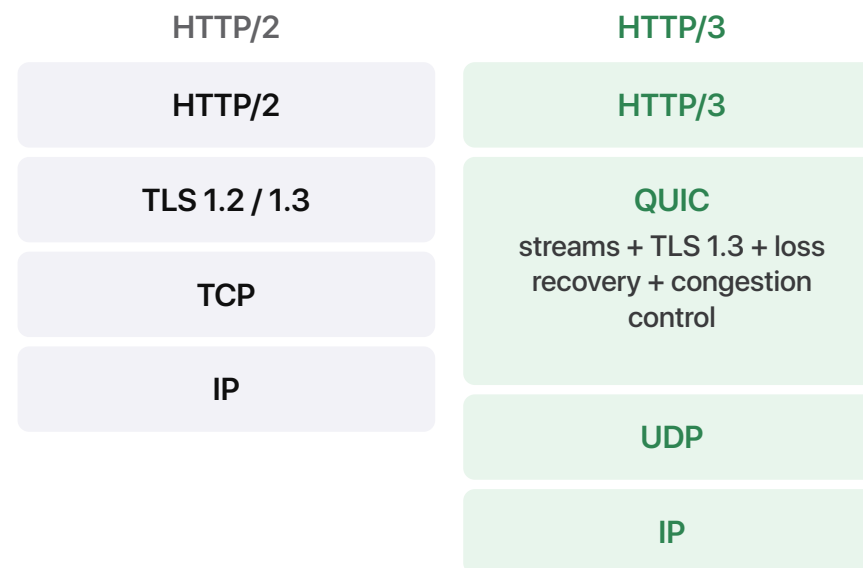
Handshake merged with TLS 1.3

Transport and encryption negotiate together: 1 RTT to first byte, 0 RTT on resumption. Encryption is not optional.

In user space

Congestion control ships with the application instead of the kernel, so it can be changed without waiting years for deployment.

WHAT MOVED WHERE



QUIC is not “HTTP on UDP”. It is a new reliable transport that happens to run on UDP because UDP is what the internet already forwards.

ENGINEERING JUDGEMENT

Where HTTP/3 Does Not Help

A network with almost no loss

Inside one datacentre, TCP head-of-line blocking rarely triggers. QUIC's main advantage disappears, and HTTP/2 keeps the kernel's optimised path.

CPU per byte is higher

Packet handling, acknowledgements and encryption happen in user space. On a saturated edge node this shows up as measurable CPU, not as free latency.

UDP gets blocked and throttled

Corporate firewalls drop UDP 443 or rate-limit it. Every client therefore needs a working TCP fallback, which means you maintain both paths.

Tooling and visibility are thinner

Encrypted transport headers mean the network team can no longer see what they used to see. Debugging moves into the application.

Sensible default in 2026

HTTP/3 at the public edge, HTTP/2 or gRPC between services, HTTP/1.1 kept alive only for clients and middleboxes that cannot do better.

SUMMARY

Three Generations Side by Side

	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC over UDP
Format	Text	Binary frames	Binary frames
Concurrency	1 per connection, ~6 sockets	Many streams, 1 socket	Many independent streams
Headers	Repeated in full	HPACK	QPACK
HOL blocking	Application and transport	Transport only	None
To first byte	2–3 RTT	2 RTT	1 RTT, or 0 resumed
Use it when	Compatibility is the requirement	Service-to-service, gRPC, clean networks	Mobile and public edge, lossy paths

04

Keeping the Connection Open

THE PROBLEM

When the Server Has News First

Request-response assumes the client knows when to ask. A price feed, a driver's position, a chat message and a build status all break that assumption.

Short polling

Ask every N seconds. Simple, cacheable, and mostly wasted: at 10 000 clients polling every 2 s you serve 5 000 RPS to say "nothing changed".

latency $\approx N/2$

Long polling

The server holds the request open until there is news or a timeout. Near-real-time, but every waiting client occupies a connection and a server-side resource.

latency ≈ 0

Server-Sent Events

One long-lived HTTP response that the server keeps writing to. Server to client only, plain HTTP, automatic reconnection built into the browser.

latency ≈ 0

WebSocket

An HTTP request upgrades into a bidirectional frame-based connection. Full duplex, minimal per-message overhead, and no HTTP semantics afterwards.

latency ≈ 0

The engineering question is never "which is newest". It is: who initiates, how often, and what does an open connection cost you per client.

ONE DIRECTION, PLAIN HTTP

Server-Sent Events

THE ENTIRE WIRE FORMAT

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
Cache-Control: no-cache

id: 42
event: price
data: {"pair":"USDKZT","bid":533.1}
```

```
id: 43
event: price
data: {"pair":"USDKZT","bid":533.4}
```

Text lines, separated by blank lines, on a response that never ends.

Reconnection is in the protocol

The client remembers the last id and sends Last-Event-ID when it reconnects. Your handler resumes from there — if you implement the resume.

It is still HTTP

Proxies, authentication, tracing headers and load balancers all work unchanged. Over HTTP/2 it is one stream among many, so the six-connection limit does not apply.

The limits

Server to client only, text only, and any proxy that buffers responses will destroy it. Client-to-server messages go as ordinary requests.

Right for

Notifications, progress, dashboards, price and status feeds — anything where the client only listens.

BOTH DIRECTIONS

WebSocket

THE UPGRADE

```
GET /ws HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: x3JJHMbDL1EzLkh9GBhXDw==
Sec-WebSocket-Version: 13
```

```
HTTP/1.1 101 Switching Protocols
```

```
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9k...
```

One HTTP request, one 101 response, and from then on the socket carries binary or text frames in both directions with a few bytes of header each.

What you gain

Full duplex, message framing, low overhead per message. The natural fit for chat, collaborative editing, multiplayer state and order entry.

What you give up

HTTP semantics. No caching, no status codes, no per-request headers — so authentication, authorisation, rate limiting and tracing all have to be designed inside your own message protocol.

What you must build yourself

Heartbeats to detect dead peers, reconnection with backoff, resume-after-gap, and a message schema with a version field.

What it does to your architecture

A long-lived connection pins a client to one instance. That is state, and state constrains deployment, autoscaling and load balancing — the next slide.

DECIDE OUT LOUD

Which One, and Why

CASE	CHOICE	BECAUSE
Chat, collaborative editing	WebSocket	Both sides send, constantly, with small messages.
Price feed, dashboard, notifications	SSE	One direction only, and you keep proxies, auth and reconnection for free.
Build status, long job progress	SSE or long polling	Rare events, and a missed one is recoverable by asking again.
Data that changes every few minutes	Short polling	Cacheable, stateless, trivially scalable. Do not pay for a connection you do not need.
Service to service, streaming results	gRPC streaming	HTTP/2 streams with a schema and code generation. Week 3.
Telemetry from many devices, no reply needed	Queue or broker	Nobody is waiting for a response, so do not hold a connection. Week 9.

BACK TO SCALING

What a Million Open Connections Cost

Memory, not CPU, is the wall

Each socket holds kernel buffers plus your per-session objects. At tens of kilobytes per connection, a million connections is tens of gigabytes before any business logic.

One thread per connection does not scale

This is the original C10K problem. The answer is event loops and non-blocking I/O — or, on modern Java, virtual threads. Week 4.

Long connections defeat load balancing

Adding an instance does not relieve the loaded ones — existing connections stay where they are. New capacity only receives new clients.

Every deploy is a reconnect storm

Restart a node and all its clients reconnect at once, all paying a handshake. Without jittered backoff on the client you have built your own denial of service.

The week 1 lesson, again

An open connection is state. Push the state out of the instance — sessions in Redis, fan-out through a broker, a connection gateway separate from business logic — and the service becomes horizontally scalable again.

PRACTICE 2 · 1 POINT

Load Testing HTTP/1.1 vs HTTP/2 vs HTTP/3

Take last week's baseline Spring Boot service. Put it behind a server that can speak all three versions, then measure the same endpoint under the same load through each one.

STEP 1

Three protocol runs

Same endpoint, same concurrency, same duration, over HTTP/1.1, HTTP/2 and HTTP/3. Report p50, p95, p99 and throughput for each.

STEP 2

The cost of the handshake

Run once with connection reuse and once with a fresh connection per request. The gap in p50 is the handshake; state it in milliseconds.

STEP 3

Add latency and loss

Shape the link (for example `tc netem` with 100 ms delay and 1 % loss) and repeat. Explain which version degrades least, and why.

Tooling **k6** or **wrk** for load, **curl --http3** for a smoke test, **nginx** or **caddy** as the front for HTTP/3, **tc netem** or Docker network settings for impairment. One client machine, one server, both stated in the report.

ACCEPTANCE CRITERIA

What Counts as Measured

ACCEPTED

A table of p50, p95, p99 and RPS for every configuration, with the load parameters and the hardware stated.

A named number for the handshake cost, taken from your own two runs.

Two or three sentences explaining each result by a mechanism from this lecture — not by “HTTP/2 is newer”.

A reproducible command line, and a README that says where AI was used.

NOT ACCEPTED

Averages only. The mean hides exactly the effect you are looking for.

A single 10-second run. Warm up first, then measure long enough for the numbers to hold still.

A load generator saturating its own CPU, so you measured your laptop instead of the server.

A result you cannot explain when asked. That is worth zero points regardless of how good the chart looks.

BEFORE WEEK 3

Reading and What Comes Next

READ

Grigorik, *High Performance Browser Networking* — the chapters on TCP, TLS and HTTP/2. The most useful hundred pages you can read this month.

RFC 9114 (HTTP/3) and RFC 9000 (QUIC) — the introductions only. Learn to read a specification; you will need the habit.

Kleppmann, *Designing Data-Intensive Applications* — chapter 1, if you have not yet.

NEXT WEEK

Networking and protocols II

REST versus gRPC versus GraphQL, Protobuf and serialisation, schema evolution and API versioning, idempotency, timeouts and retries.

gRPC runs on the HTTP/2 streams and framing from today, so bring today's vocabulary with you.

Practice submissions and questions in MS Teams. Office hours as announced.